

To appear in *Quantitative Finance*, Vol. 00, No. 00, Month 20XX, 1–14

Deep Implicit Layers for No-Arbitrage Implied Volatility Surface Interpolation

R. ZIMMER[†]

[†]Institute of Mathematics and Computer Sciences, University of São Paulo, São Paulo, Brazil

(Received 00 Month 20XX; in final form 00 Month 20XX)

We implement a neural network architecture based on deep implicit layers with the objective of optimizing the parameters of multiple SABR (stochastic alpha, beta, rho) models in the context of financial options. Our network is trained on a dataset of historical traded option contracts and their respective calculated implied volatility values. We perform a candidate selection process using summarized suitability scores to remove possible outliers from our candidate set, aiming to minimize arbitrage opportunities when optimizing for the SABR model parameters. Our approach leverages the implicit formulation for the scoring process that solves for an implicit fixed-point problem between the observed data and SABR generated surfaces, where a transformer architecture is combined to deep implicit layers to process sequences of contracts with varying lengths. The surfaces produced with our scoring network are then compared to a baseline linear interpolation of the daily surfaces.

Keywords: Neural Networks; Implied Volatility; Transformers; Quantitative Methods; Implicit Layers; Equity Options

JEL Classification: C45; G13; C63; C61

1. Introduction

1.1. Options and Implied Volatility

Option contracts are one of the most important types of derivative contracts in the context of financial markets. A derivative is a contract traded with respect to another asset (also called the underlying asset) and an option contract allows the buyer of the contract to sell or buy the specified asset from the seller at a predefined future date. The pricing equations for these derivatives, such as the Black-Scholes model (Black and Scholes 1973), typically require knowing the time to maturity of the contract, the price of the underlying asset to be traded, the market's current risk-free rate, and the price at which the asset will be executed at maturity, also called strike price (Hull 2018). The underlying asset's volatility is also an important parameter when calculating the option's price, but knowledge of its true future value is not possible, so typically a market estimate of the future volatility is used. This estimate, called the implied volatility (IV or σ_{iv}) is obtained by numerically solving for the volatility in the pricing equation using market quoted prices.

Having access to the implied volatility values allows investors to make sales at prices consistent with market expectations, that is, the no-arbitrage price for the contract assuming an efficient market (Hull 2018). However, the IV values corresponding to all strike prices and maturities are not directly observable, as not all possible contract combinations are traded daily. Ensuring

Corresponding author. Email: rzimmerdev@gmail.com

a way to obtain these values with confidence and using only readily available data is a crucial task for quantitative analysts (Kim *et al.* 2006). Usual strategies for estimating these values are predominantly based on linear or spline interpolation algorithms (Homescu 2011) or stochastic differential equation models (Jordan and Tier 2011) that use numerical methods to approximate the values from existing listed contracts. Most approaches approximate values for the implied volatility using pairs of time to maturity and strike price as variables. This allows visualizing the chosen model results in the form of a surface, where the $\mathbf{XY}$ axes are the maturity and strike pairs and the $\mathbf{Z}$ axis is the IV . This surface, called the **implied volatility surface** (IVS), denoted by $\mathcal{S}$ is therefore a function of the maturity time t and the strike price K ,

$$\mathcal{S} := f(t, K), \quad \text{where } f: \mathbb{R}^2 \rightarrow \mathbb{R}. \quad (1)$$

Ensuring accurate predictions of the IVS is crucial, as even small errors can lead to incorrect pricing and consequently large losses of profit or even negative return (Hagan *et al.* 2014) given the leveraging capabilities of options when compared to non-derivative securities. Classic models such as linear interpolation, polynomial fitting, and *splines* also lack flexibility for extrapolation and often generate either non-smooth surfaces, arbitrage opportunities, or both (Hagan *et al.* 2014).

1.2. Contributions of the Presented Work

In this paper, we address the challenges of interpolation and extrapolation of the IVS, focusing on reducing arbitrage opportunities. The chosen method uses a stochastic differential equation solution approach, specifically the *Stochastic Alpha, Beta, Rho* (*SABR*) model. The parameters α , β , ρ , and ν of the *SABR* model are obtained through the Broyden-Fletcher-Goldfarb-Shanno (BFGS) quasi-Newton non-linear optimization algorithm (Broyden 1970, Fletcher 1970, Goldfarb 1970, Shanno 1970). This step is commonly performed using only a subset of the daily observed prices (Kim *et al.* 2021), due to the fact that some points may contain unreliable prices of illiquid contracts that do not match the real prices of the derivative, and their usage in the optimization step increases the variance of the final model. To overcome this initial difficulty, the points used as inputs to the BFGS algorithm are often heuristically filtered by human experts (Hagan *et al.* 2002). Instead of manually selecting these candidate points, models capable of automatically receive point clouds and returning a numerical score to be used as selection criteria — the implicit volatility values directly — such as transformer architectures, have already shown promising results (Cao *et al.* 2020, Kim *et al.* 2021, Bae *et al.* 2024, Jerbi and Chaabene 2020).

The main contribution of this work will be the adaptation of *neural implicit layers*, a physics-inspired variation of fully connected linear layers where layer weights are optimized to solve for a fixed point equation, to be used in conjunction with a transformer architecture (Vaswani *et al.* 2017) for the task of filtering candidate points for the *SABR* parameter optimization. This approach is an implicit formulation variation for the scoring model proposed by Kim *et al.* (2021), aiming to enforce a zero arbitrage condition in the resulting generated surface when solving the fixed point equation. This type of layer has been shown to effectively represent smooth surface densities and demonstrated state-of-the-art results in function fitting tasks similar to the one presented in this paper (Kawaguchi 2021), all while being computationally faster for solving interpolation problems than linear layer models with similar performance (Kolter *et al.* 2020). Additionally, transformers have also been shown to perform better at handling unordered point clouds over time compared to recurrent neural layers (Kim *et al.* 2024) for IVS interpolation tasks, making them a more suitable choice for the task at hand. While directly fitting a neural network to the IVS is possible, optimizing the parameters for a domain-specific model in the context of IVS interpolation has been shown to be more effective (Almeida *et al.* 2023) due to the necessity of enforcing smoothness and reducing arbitrage opportunities.

1.3. The Transformer Architecture

The aforementioned *SABR* model is a parametric stochastic volatility approach for obtaining smooth volatility surfaces and the transformer architecture is used as the scoring model to filter the candidate points for the optimization process. This score, called the *summarized suitability* score (*SS* score), is assigned to each daily traded options contract, and indicates how adequate a candidate point is at summarizing the format of the resulting surface. It measures how much removing a candidate point would affect the generated surface shape and smoothness¹. The highest scoring points — which best represent the surface — are then selected and used as input for the BFGS algorithm used to fit the *SABR* model parameters. This process aims to remove outlier points that might introduce variance, and consequently arbitrage opportunities (Kim *et al.* 2021), in the optimization process for each of the *SABR* models².

1.4. Fixed-Point problems

Differently from the usual function approximation tasks, a fixed point problem is instead defined by conditions that the output must satisfy. For explicit functions, a given output z is computed directly from the input x using a function f , such that $z = f(x)$. In contrast, an implicit function g is such that it depends on both the input x and the expected output z , where z is a solution for the equation:

$$g(x, z) = 0 \tag{2}$$

Formulating the problem as a fixed point equation can be used for a multitude of problems, including finding fixed points in recurrent networks, solutions to differential equations leading to neural ordinary differential equations, and several physics optimization problems (Chen *et al.* 2019). The implicit formulation offers several practical advantages, primarily the separation of the solution method from the layer definition which can be useful in maintaining interpretability of the resulting model (Kawaguchi 2021) and as a way to enforce constraints in the optimization process, the latter being the main focus of this work.

In our case, g is the mean absolute error between the observed implied volatility values for traded contracts and the actual interpolated implied volatility values for the same strike-maturity pairs. If we use the *SABR* model as the underlying function to interpolate the implied volatility values we can additionally be sure that the generated surface is smooth and mostly arbitrage free (Hagan *et al.* 2002, Grzelak and Oosterlee 2016, Hagan *et al.* 2014). Thus, by finding the set of *SABR* parameters x that solve the equation $g(x, z) = 0$ we can reduce arbitrage opportunities for the resulting interpolated IV values.

Moreover, the implicit formulation can result in significant benefits when used in deep learning and automatic differentiation frameworks (Massaroli *et al.* 2021, Kolter *et al.* 2020). Automatic differentiation methods store the entire computation graph, which can be memory-intensive. With implicit layers, however, the implicit function theorem allows computing gradients directly at the solution point and therefore storing intermediate variables becomes redundant (Massaroli *et al.* 2021). This improves memory efficiency, making implicit layers particularly advantageous when used to substitute fully-connected output layers within large deep learning models (Li *et al.* 2020).

¹The use of a per-point score can be compared to the principal components of a collection of points within the context of Principal Component Analysis (PCA), a technique commonly used to reduce the dimensionality of input features.

²A separate model is fitted for each unique strike price in a given day. Section 2.1 covers additional details on fitting the *SABR* model at each unique strike price.

1.5. Anderson Acceleration for Implicit Layers

The Anderson acceleration (AA) method is a technique used to accelerate the convergence of fixed-point iterations. It is particularly useful when the fixed-point iteration is slow to converge or when the fixed-point iteration is used in a deep learning context. Instead of using the current fixed-point iteration as the next guess, as in standard fixed-point iteration, a linear combination of the previous guesses is used to extrapolate the next, according to the following expression:

$$x_{k+1} = \beta_k x_k + (1 - \beta_k) x_{k-1}$$

where $\beta_k > 0$ is the mixing factor (also called relaxation parameter) used to determine the extrapolation weight. Two additional hyperparameters are used: the damping factor λ used to stabilize the fixed-point iteration; and a memory size m that determines the number of previous computed points to use in the extrapolation. Our proposed architecture applies the AA method as shown in Algorithm 1 to solve a fixed-point equation within the implicit layers, and the layer weights are adjusted afterward via backpropagation.

Algorithm 1 Equilibrium Model with Anderson Acceleration

Require: Function f , initial point x_0 , memory size m , damping factor λ , maximum iterations K , tolerance ϵ , mixing factor β

- 1: Initialize $X_0 = x_0, X_1 = F_0 = f(x_0), F_1 = f(F_0)$
- 2: **for** $k = 2$ to K **do**
- 3: $j \leftarrow \min(k, m)$
- 4: $G = F_{0,\dots,j} - X_{0,\dots,j}$
- 5: $H = GG^\top + \lambda I$
- 6: Solve $H^{-1} \cdot y = \alpha$
- 7: $\alpha \leftarrow \alpha_{1,\dots,n+1}$
- 8: $X_k \leftarrow \beta \alpha^\top F_0 + (1 - \beta) \alpha^\top X_0$
- 9: $F_k \leftarrow f(X_k)$
- 10: Compute residual:
 $\text{residual} = \frac{\|F_k - X_k\|}{\|F_k\| + 10^{-5}}$
- 11: **if** residual $< \epsilon$ **then**
- 12: **break**
- 13: **end if**
- 14: **end for**
- 15: **return** X_k

The final deep equilibrium model used within the implicit layer (Algorithm 2) uses the forward pass of a linear layer as input to the Anderson acceleration algorithm, where the solution to a fixed-point equation is computed. This numerical solution can then be used as input for the next layer, or as the final solution to the fixed-point equation itself if the layer is last in the network. When generating the IVS, the output of a transformer encoder is used as input for the implicit layer, and a mean absolute error (MAE) function between the observed surface and SABR-generated surface serves as target function within the fixed point iteration to minimize arbitrage opportunities for listed options in the generated surface.

Algorithm 2 Implicit layer forward pass with Anderson acceleration (AA).

```

1: function IMPLICITLAYER( $x, m, \lambda, \epsilon, \beta$ )
2:   function  $f(x) \leftarrow W \cdot x + b$ 
3:    $x \leftarrow \text{AA}(f, x, m, \lambda, \epsilon, \beta)$ 
4:    $y \leftarrow x \cdot W^T + b$ 
5:   return  $y$ 
6: end function

```

2. Implementation of a Parametric SABR Model and Implicit Neural Network Architectures

2.1. Defining the SABR Model Equations

The SABR model is a stochastic differential model, defined by the following equations in which the dynamics of the asset price and the corresponding volatility levels are used to calculate an approximation to the implied volatility where no contract is traded.

- **Alpha** (α): This is the initial volatility level.
- **Beta** (β): The elasticity parameter between the volatility of the asset and its price.
- **Rho** (ρ): Statistical correlation between the asset price and its volatility, also called *spot-volatility correlation*.
- **Volvol** (ν): The volatility of the volatility process itself.

The simplified function for the implied volatility using the SABR model and logarithmic “mon-eyness” $x = \log\left(\frac{K}{F_t}\right)$ is then given by Hagan *et al.* (2002):

$$\sigma_{BS}(F_t, K) = \alpha \frac{K^{\frac{1-\beta}{2}}}{1 + \left(\frac{(1-\beta)^2 \rho^2}{24} + \frac{(1-\beta)^2 (2-3\rho^2)}{1920} \right) (F_t K)^{1-\beta}}, \text{ where } F_t = S e^{(r-q)t}$$

and S is the underlying asset price, r is the risk-free rate, q is the dividend yield, t is the time to maturity, K is the strike price and F_t is the forward price of the asset for a given time to maturity. The equation presented above corresponds to the simplified version of the equation, which does not contain all specifiable parameters. Refer to Appendix A for the full equation which is used in the actual implementation.

In our approach, we optimize one set of SABR parameters for each unique strike price per day. Three continuous functions map from continuous maturities to the model parameters α , ρ , and ν . This ensures a smooth transition between the different strike levels, as using a single choice for the parameters for all strike and day pairs alone can introduce discontinuity in the resulting implied volatility surfaces (Hagan *et al.* 2015, Kim *et al.* 2021). Given the functions for the parameters $\alpha_t = f_\alpha(t, \mathbf{P})$, $\rho_t = f_\rho(t, \mathbf{Q})$, and $\nu_t = f_\nu(t, \mathbf{R})$ ¹, and the encoding variables $\mathbf{P}$, $\mathbf{Q}$, and $\mathbf{R}$ respectively, such that $\mathbf{P} \in \mathbb{R}^5$, $\mathbf{Q} \in \mathbb{R}^4$, and $\mathbf{R} \in \mathbb{R}^4$. The functions for the parameters can then be expressed in terms of the encoding variables and continuous time to maturity t as follows:

$$f_\alpha(t, \mathbf{P}) = P_0 + \frac{p_3}{P_4} \left(\frac{1 - e^{-P_4 t}}{P_4 t} \right) + \frac{P_1}{P_2} e^{-P_2 t}$$

¹The value for β corresponds to the sensitivity of the forward price towards movements in the spot price, and represents a characteristic of the asset itself. As it does not change significantly over time to justify additional calibration, we treat it as a constant hyperparameter instead.

$$f_\rho(t, \mathbf{Q}) = Q_0 + Q_1 t + Q_2 e^{-Q_3 t}$$

$$f_\nu(t, \mathbf{R}) = R_0 + R_1 t^{R_2} e^{R_3 t}$$

Therefore, the task simply becomes obtaining the encoding variables $\mathbf{P}$, $\mathbf{Q}$, and $\mathbf{R}$ for each daily observed data points. We use the Broyden-Fletcher-Goldfarb-Shanno (*BFGS*) algorithm to optimize the encoding variables for minimum mean absolute error (MAE) between the observed implied volatilities and corresponding SABR-generated implied volatilities, in other words, we chose $\mathbf{P}$, $\mathbf{Q}$, and $\mathbf{R}$ which best represent the observed daily surface. Additionally, using a Monte Carlo estimation of only a subset of the daily candidate set, we calculate summarized values for the encoding parameters, represented by $\tilde{\mathbf{P}}$, $\tilde{\mathbf{Q}}$, and $\tilde{\mathbf{R}}$. A more detailed explanation of the estimation process is given in Section 2.2.3.

To get an actual estimate of how much each of the observed data points contribute to the observed surface, we calculate a separate MAE score, called the summarized suitability (SS) score (Kim *et al.* 2021) for the values $f_\alpha(t, \mathbf{P})$, $f_\rho(t, \mathbf{Q})$, $f_\nu(t, \mathbf{R})$ versus the values for $f_\alpha(t, \tilde{\mathbf{P}})$, $f_\rho(t, \tilde{\mathbf{Q}})$, and $f_\nu(t, \tilde{\mathbf{R}})$.

This process is similar to how a human trader would manually decide which data points were most relevant in generating the observed surface, or rather, which data points best summarize the general shape of the surface. Thus, knowing the SS scores facilitates can guide the removal of outliers by filtering out data points with SS scores below a certain threshold or quantile when sorting in ascending order. Removing outliers before fitting for the SABR model parameters is done to reduce arbitrage between market observed and interpolated option contract prices (Kim *et al.* 2021).

A visual representation of a surface obtained without selecting any data points for removal and thus containing an arbitrage opportunity is shown in Figure 1. When no selection is performed to remove outliers, the resulting SABR model can generate surfaces with clear peaks, which can result in arbitrage opportunities if a contract where to be sold or bought for the corresponding price according to the Black-Scholes(Black and Scholes 1973) price. Thus, by applying a selective filtering criteria, such as the Monte Carlo SS score estimate, we can greatly reduce arbitrage opportunities. Additionally, if we make sure no peaks or differences between the actually SABR-generated values and observed values exist, we can be certain no arbitrage opportunity will be present with regard to the market-observed strike and maturity pairs To solve this problem, we use a transformer-based neural network architecture to greatly increase processing the processing speed for the estimation of the aforementioned SS scores, while inserting implicit equilibrium layers at the output of the network to reduce the errors the SABR generated surfaces have with respect to actual observed implied volatilities.

2.2. Proposed Architecture

As mentioned, transformers are capable of capturing complex dependencies in sequential non-fixed length data. Within our proposed architecture, shown in Figure 2, we use a transformer encoder to approximate the SS scores for each candidate point $\mathcal{S}_i^t \in \mathcal{S}^t$ from the daily observed input.

2.2.1. Details on the Chosen Transformer Encoder. Unlike recurrent neural networks (RNNs) and convolutional neural networks (CNNs), transformers do not rely on a fixed-length data sequence or fixed window size (Wen *et al.* 2023). Instead, they use an attention mechanism, which allows each input element (or token) to consider the relationship with all other elements in the same sequence simultaneously (Vaswani *et al.* 2023). This is achieved through a block of layers using attention heads.

The attention mechanism accepts continuous data and calculates a value for different heads through the value (V), key (K), and query (Q) matrices, resulting in an output weighted by the

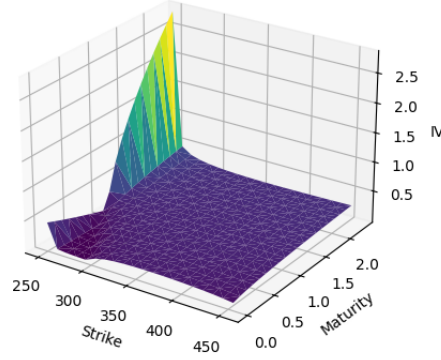


Figure 1.: Interpolated surface using SABR models fit on a unfiltered daily candidate set.

importance of each element in the sequence. Specifically, for our problem we use the mechanism of self-attention, and therefore the three matrices are equal.

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right) V \quad (3)$$

Transformers are classified as machine learning algorithms due to their ability to learn patterns and relationships from data, and were originally proposed for natural language processing tasks. In the context of our specific architecture, we use only the encoder block and adapt it to perform a regression on the SS scores. The hidden values are passed to fully-connected (FC) layers, and implicit fixed-point iteration layers are added as final output layers.

2.2.2. Details on the Implicit Layer. Using a Deep Equilibrium Model variation for the Implicit Layer, we adapt Equation (2) by using the following fixed point equation instead, where g is the MAE between the observed surface and the generated surface using the SABR model optimized on the filtered candidate set.

$$\begin{aligned} g\mathbf{W}(x, z^*) &= 0 \\ z^* &= f(z^*, x) \end{aligned} \quad (4)$$

where $\mathbf{W}$ are the layer weights and biases, f is an activation function, and z^* is the target output of a batched non-normalized Jacobian to be used as the gradient function in our code, given by the equation below. Finally, y are the hidden values as well as the final output scores.

$$\left(\frac{\partial z^*(\cdot)}{\partial(\cdot)}\right)^T y = \left(\frac{\partial f(z^*, x)}{\partial(\cdot)}\right)^T g\mathbf{W}$$

2.2.3. Monte Carlo Estimation of SS Scores. As mentioned in Section 2.1, a Monte Carlo process is used to estimate synthetic target SS scores for the model. Instead of using the entire daily candidate set for calculating the encoded variables $\mathbf{P}$, $\mathbf{Q}$, and $\mathbf{R}$ for the SABR model, only k subsets with size $p \cdot n$, where n is the total number of candidate points and p is a percentage

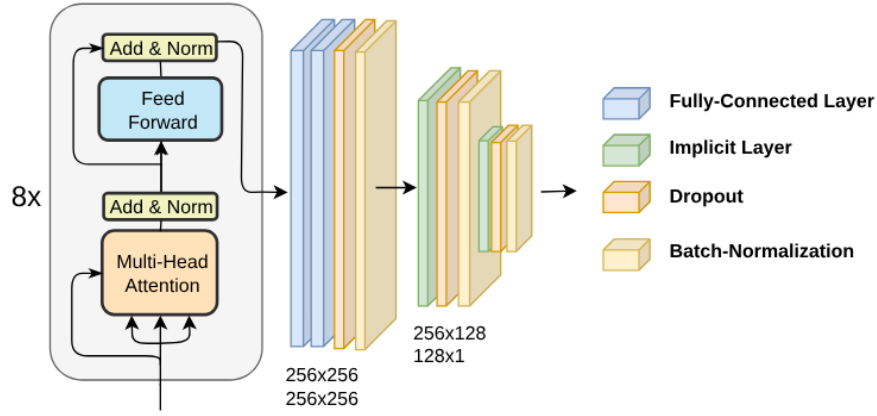


Figure 2.: Visualization of the Transformer Encoder and Implicit Layers used in the proposed architecture.

value, of the candidate set are used as input to the *BFGS* algorithm to reduce surface volatility, resulting in $\tilde{\mathbf{P}}$, $\tilde{\mathbf{Q}}$, and $\tilde{\mathbf{R}}$. The SS score is then calculated as the mean absolute error (MAE) between the functions f_α , f_ρ , and f_ν using both encoded variable types as shown in Algorithm 3 and its normalized values are used as target values for our model. The choice of using a neural network approach for calculating the SS scores is motivated by the much higher computational cost of the Monte Carlo estimation.

The Monte Carlo estimation is performed with a small finite number of subsets due to computational constraints, both in terms of time and memory. Finally, model training is performed until convergence to low RMSE values between the scores predicted with the network and the ones generated by the Monte Carlo estimation. After training, the model prediction substitutes the Monte Carlo estimation for the SS scores, and the best candidate set is selected and passed to the *BFGS* algorithm to obtain the final encoding $\mathbf{P}_{nn}$, $\mathbf{Q}_{nn}$, $\mathbf{R}_{nn}$ variables for the SABR models to be used for the actual surface interpolation.

Algorithm 3 Monte Carlo Estimation of normalized Target SS Scores

Require: Daily candidate set $\mathcal{S}^t$, percentage $p = 0.8$, number of subsets $k = 1 \times 10^2$

- 1: $\alpha^*, \rho^*, \nu^* \leftarrow BFGS(\mathcal{S}^t)$
 - 2: $SS_\alpha, SS_\rho, SS_\nu \leftarrow \mathbf{0}$
 - 3: **for** candidate c in $\mathcal{S}^t$ **do**
 - 4: $\mathcal{S}_i^t \leftarrow \text{Sample}(\mathcal{S}^t, k, p)$
 - 5: $\tilde{\mathbf{P}}, \tilde{\mathbf{Q}}, \tilde{\mathbf{R}} \leftarrow BFGS(\mathcal{S}_i^t)$
 - 6: $SS_{\alpha,c} \leftarrow \text{MAE}(f_\alpha(t, \tilde{\mathbf{P}}) - \alpha^*)$
 - 7: $SS_{\rho,c} \leftarrow \text{MAE}(f_\rho(t, \tilde{\mathbf{Q}}) - \rho^*)$
 - 8: $SS_{\nu,c} \leftarrow \text{MAE}(f_\nu(t, \tilde{\mathbf{R}}) - \nu^*)$
 - 9: **end for**
 - 10: z-normalize $SS_\alpha, SS_\rho, SS_\nu$
-

3. Methodology and Results

3.1. Data Description and Pre-Processing

The dataset used for training was obtained from historical traded option contracts on the *S&P 500 ETF*¹ from January 2021 to December 2022, with a total of 68 unique days. The dataset contains various columns on traded options, indexed by date, including strike prices, future contract prices, maturities, implied volatilities, among others. Relevant to our study, the principal columns used were:

- **Underlying:** Price of the underlying asset.
- **Strike:** Option's strike price.
- **Expiration:** Days remaining until the option's expiration.
- **Implied Volatility:** Value for the implied volatility.
- **Date:** Traded date for the contract, used as the preprocessed dataset index.

Additionally, two columns were added to be used for the Monte Carlo estimation:

- **Maturity:** Calculated by dividing *Expiration* by 252 (working days in a year) to normalize the maturity time.
- **Risk-free rate:** Risk-free rate obtained from the Federal Reserve Economic Data (FRED) for the corresponding period.
- **Dividend:** Dividend yield paid quarterly by the SPY index.

3.2. Data Preprocessing

The data preprocessing involved several essential steps, predominantly to allow the use of the surface data directly in the transformer architecture and subsequent generation of the surface for historical data. Each of the trained transformers (one for each parameter α , ρ , or ν) takes four dimensions as input, which are:

- (i) Time to maturity;
- (ii) The daily parameters α , ρ , or ν calculated using the *BFGS* optimization algorithm;
- (iii) **1** if the parameter was calculated for the current day, **0** if it was calculated for a fixed set of maturities (top 7 most frequent maturities) using the previous day fitted parameters;
- (iv) Parameter value calculated using the previous day's **P**, **Q** and **R** values for same strike and maturity.

The target scores were generated before training using the Monte Carlo algorithm, detailed in Section 2.2.3. with $p = 80\%$ and $k = 1 \times 10^2$. For training and testing we also observed that the choice of $\beta = 0.5$ performed best at minimizing the surface's MAE.

3.3. Results and Surface Comparison

A baseline linear interpolation for the daily observed points was also used to compare our trained model's surfaces by interpolating the points within a fixed grid of strikes and maturity. No extrapolation was performed due to the nature of the linear model.

The training dataset had a total of 44 817 observations totaling only 68 unique days, which lead to the decision of using a leave-one-out cross-validation approach. The Adam algorithm was chosen for optimization, with a starting learning rate of 1.0×10^{-5} , a max epoch limit of 10^3 and an early stopping sensitivity of 10^{-3} .

¹An exchanged-traded fund that replicates the S&P500 index, primarily listed on NYSE Arca.

A batch size of 8 was used to avoid memory issues when training the implicit layer. The network architecture is a transformer encoder block with 8 sequential attention heads, 2 linear layers with 256 hidden units and 2 implicit layers with 256 and 128 hidden units, respectively, and a dropout rate of 0.1 was used for the dropout layers. An illustration of the architecture is shown in Figure 2.

Finally, the Anderson acceleration as described in Algorithm 1 was implemented with a maximum iteration limit of $K = 10$ steps, a convergence threshold of $\epsilon = 10^{-4}$, damping factor of $\lambda = 0.1$, and memory size of $m = 5$.

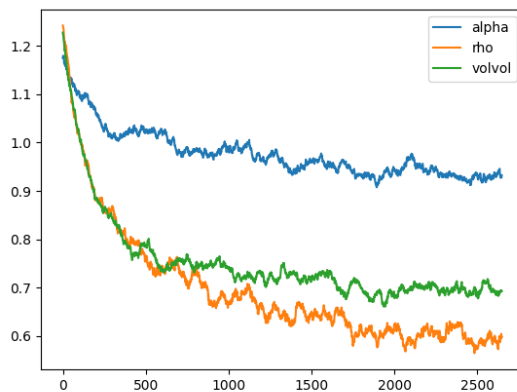


Figure 3.: Historical loss curve on training session data. Curves were cut for better visualization, with the largest episode being 2950 epochs for the SST_α (alpha) network.

Model	Test loss (RMSE)	Train time	Preprocess time (Monte Carlo Estimation)
SST_α	1.21	4m21s	7m50s
SST_ρ	0.72	2m8s	6m34s
SST_ν	0.75	1m43s	6m41s
Total	-	8m12s	21m5s

Table 1.: Model training metrics for each of the three SST models.

For benchmarking, the execution time was measured using the Python `perf_counter` method and a heat-up of the model was performed beforehand on the entire dataset. Separate tests using a linear interpolation with the original points and with the implicit layer were performed to compare both generated surfaces, shown in Figure 4. The linear interpolation of course obtained a MAE of 0.0 to the observed surface, and the proposed model had a MAE of 7×10^{-4} for the points used in the Monte Carlo estimation. While still near the threshold hyperparameter of 1×10^{-4} , a tradeoff could be made to decrease the MAE between the interpolated IVs and actual market observations by increasing the iteration limit K for the Anderson acceleration. This would, in turn, greatly decrease both training and execution speeds.

The mean train and test losses using LOO cross-validation are shown in Table 1, while the individual loss curves for each of the individual selector networks responsible for the f_α , f_ρ , and f_ν functions are shown in Figure 3. The execution times shown below in Table 2 reflect the wall time for testing on the entire dataset and on one single trading day (data point)¹. The obtained RMSE

¹We were unable to determine why the wall time for testing on the entire dataset was 229.1ms, noticeably higher than the expected $68 \times 2\text{ms} = 136\text{ms}$, but we attributed this difference to accumulated overhead associated with sending and retrieving data from the GPU multiple times.

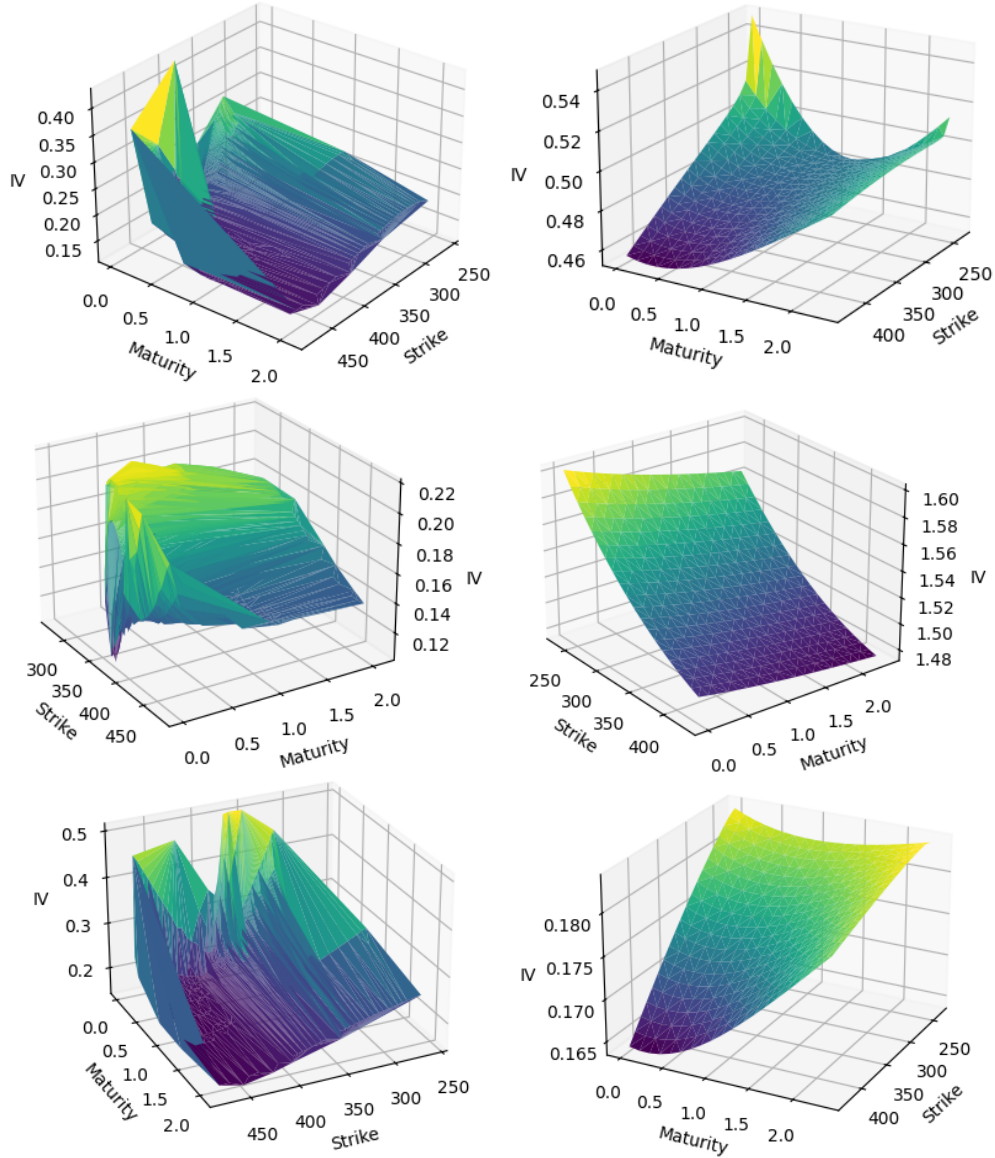


Figure 4.: Surfaces interpolated on test data using a linear interpolation model (left) compared to the SABR models fitted with the candidate set selected using the trained implicit transformer architecture (right).

values for the test data show that the model was able to generalize well to the test data, with the lowest RMSE value for the ρ parameter.

Metric	Mean time	Standard Deviation
Execution time on entire dataset	229.1ms	41.2ms
Execution time per point	2ms	1ms

Table 2.: Model performance

4. Conclusion

4.1. Summary

This work presents an architecture drawing on approaches by Kim *et al.* (2021) and Kolter *et al.* (2020) to optimize the parameters of multiple SABR models using a neural network with implicit deep equilibrium layers as the candidate set selection mechanism. Our approach was successful in enforcing the no-arbitrage conditions in the resulting surfaces by solving for a fixed-point problem between the observed data and generated IV values. The time complexity of the trained model vastly outperformed the Monte Carlo method, with a speedup of 6300%, and was able to generalize well to test data, as shown in Table 1 and Table 2, respectively. Additionally, the neural network approach supports fine-tuning to incorporate new daily observations; a significant advantage over the Monte Carlo method, which would require estimation from scratch using the new dataset.

4.2. Implications and Future Work

The main focus of this work was to reduce arbitrage conditions by reducing the differences between the surfaces generated with the SABR models and actual surfaces observed from daily traded option contracts. We leveraged the ability of the implicit formulation in solving for a fixed-point problem, combined with the very low execution times that were only possible with the Anderson acceleration method, ensuring near optimal no-arbitrage conditions between the generated surfaces and observed implied volatility surfaces.

The approach of using neural networks, even though already significantly faster than Monte Carlo methods, could be further optimized in the context of high-frequency intraday trading. Adapting the feature set to include historical exogenous market factors, besides just traded contracts, such as macroeconomic indicators, market indexes, and other financial instruments, could allow the model to better adapt to changing market regimes and could be a promising extension in future research.

Appendix A: Complete Implied Volatility Equation for the SABR model

The SABR model is a stochastic volatility model developed as an extension of the Black-Scholes model that allows for the volatility of the asset to be stochastic, and is widely used in the financial industry for option pricing and risk management. The implied volatility of the asset according to the model is given by the following equations (Hagan *et al.* 2002):

$$F_t = S \cdot e^{(r-d)t}$$

$$x(t) = \log \left(\frac{F_t}{K} \right)$$

where F_t is the price of a forward contract with the same underlying asset as the option, S is the spot price of the asset, r the risk-free rate, and d the dividend yield. $x(t)$ is the log-moneyness

value for the option.

$$\begin{aligned} y(t) &= (F_t K)^{\frac{1-\beta}{2}} \\ z(t) &= \frac{\nu}{\alpha} x(t) y(t) \\ \kappa(t) &= \log \left(\frac{\sqrt{1 - 2\rho z(t) + z(t)^2} + z(t) - \rho}{1 - \rho} \right) \end{aligned}$$

where y and z are auxiliary functions to simplify the SABR model equation, and κ represents the adjustment factor for the volatility. Finally, we have the complete equation corresponding to the implied volatility σ :

$$\sigma(t, K) = \alpha t \frac{\left(1 + \left(\frac{(1-\beta)^2 \alpha^2}{24 y(t)^2} + \frac{\alpha \beta \rho \nu}{4 y(t)} + \frac{(2-3\rho^2) \nu^2}{24} \right) \right)}{\left(1 + \frac{(1-\beta)^2 x(t)^2}{24} + \frac{(1-\beta)^4 x(t)^4}{1920} \right)} \frac{z(t)}{y(t) \kappa(t)}.$$

As mentioned, the α, β, ρ, ν parameters are usually obtained through constrained optimization, and algorithms such as adaptations of the Newton-Raphson method or Sequential Quadratic Programming can be used instead of the BFGS algorithm. Nonetheless, discussions regarding the best choice for an optimization algorithm for this purpose is outside the scope of this work.

Funding Statement

No funding was received

Disclosure of interest

No potential conflict of interest was reported by the author.

References

- Shanno, D.F., Conditioning of quasi-Newton methods for function minimization. *Mathematics of Computation*, 1970, **24**, 647–656.
- Broyden, C.G., The convergence of a class of double-rank minimization algorithms 1. general considerations. *IMA Journal of Applied Mathematics*, 1970, **6**, 76–90.
- Goldfarb, D., A family of variable-metric methods derived by variational means. *Mathematics of Computation*, 1970, **24**, 23–26.
- Fletcher, R., A new approach to variable metric algorithms. *Computer Journal*, 1970, **13**, 317–322.
- Black, F. and Scholes, M., The Pricing of Options and Corporate Liabilities. *Journal of Political Economy*, 1973, **81**, 637–654.
- Hagan, P., Kumar, D., Lesniewski, A. and Woodward, D., Managing Smile Risk. *Wilmott Magazine*, 2002, **1**, 84–108.
- Kim, B.H., Lee, D. and Lee, J., Local volatility function approximation using reconstructed radial basis function networks. In *Proceedings of the ADVANCES IN NEURAL NETWORKS - ISSN 2006, PT 3, PROCEEDINGS*, edited by J. Wang, Z. Yi, J. Zurada, B. Lu and H. Yin, Vol. 3973 of *Lecture Notes in Computer Science* 3rd International Symposium on Neural Networks (ISSN 2006), Chengdu, PEOPLES R CHINA, MAY 28-31, 2006, pp. 524–530, 2006, SPRINGER-VERLAG BERLIN.

- Jordan, R. and Tier, C., Asymptotic Approximations to CEV and SABR Models. *SSRN Electronic Journal*, 2011, pp. 1–38 Available at SSRN: <https://ssrn.com/abstract=1850709> or <http://dx.doi.org/10.2139/ssrn.1850709>.
- Homescu, C., Implied Volatility Surface: Construction Methodologies and Characteristics. , 2011.
- Hagan, P.S., Kumar, D., Lesniewski, A. and Woodward, D., Arbitrage-Free SABR. *Wilmott*, 2014, pp. 60–75.
- Hagan, P., Lesniewski, A. and Woodward, D., Vol. 110In *Probability Distribution in the SABR Model of Stochastic Volatility*, pp. 1–35, 2015.
- Grzelak, L.A. and Oosterlee, C.W., From Arbitrage to Arbitrage-Free Implied Volatilities. *Journal of Computational Finance*, 2016, **20**, 1–19 Last revised: 5 Jul 2016.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L. and Polosukhin, I., Attention is All You Need. In *Proceedings of the Proceedings of the 31st International Conference on Neural Information Processing Systems*, pp. 5998–6008, 2017, Curran Associates Inc.
- Hull, J.C., *Options, Futures, and Other Derivatives*, 10th , 2018, Pearson Education Limited.
- Chen, R.T.Q., Rubanova, Y., Bettencourt, J. and Duvenaud, D., Neural Ordinary Differential Equations. , 2019.
- Kolter, Z., Duvenaud, D. and Johnson, M., Deep Implicit Layers - Neural ODEs, Deep Equilibrium Models, and Beyond. NeurIPS 2020 tutorial. Retrieved from <https://implicit-layers-tutorial.org/>, 2020.
- Jerbi, Y. and Chaabene, S., European call price modelling using neural networks in considering volatility as stochastic with comparison to the Heston model. *Journal of Statistical Computation and Simulation*, 2020, **90**, 1793–1810.
- Cao, J., Chen, J. and Hull, J., A neural network approach to understanding implied volatility movements. *QUANTITATIVE FINANCE*, 2020, **20**, 1405–1413.
- Li, X., Wong, T.K.L., Chen, R.T.Q. and Duvenaud, D., Scalable Gradients for Stochastic Differential Equations. , 2020.
- Kim, H., Park, K., Jeon, J., Song, C., Bae, J., Kim, Y. and Kang, M., Candidate point selection using a self-attention mechanism for generating a smooth volatility surface under the SABR model. *EXPERT SYSTEMS WITH APPLICATIONS*, 2021, **173**.
- Massaroli, S., Poli, M., Park, J., Yamashita, A. and Asama, H., Dissecting Neural ODEs. , 2021.
- Kawaguchi, K., On the Theory of Implicit Deep Learning: Global Convergence with Implicit Layers. , 2021.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L. and Polosukhin, I., Attention Is All You Need. , 2023.
- Almeida, C., Fan, J., Freire, G. and Tang, F., Can a Machine Correct Option Pricing Models?. *JOURNAL OF BUSINESS & ECONOMIC STATISTICS*, 2023, **41**, 995–1009.
- Wen, Q., Zhou, T., Zhang, C., Chen, W., Ma, Z., Yan, J. and Sun, L., Transformers in Time Series: A Survey. , 2023.
- Bae, H.O., Kang, S. and Lee, M., Option Pricing and Local Volatility Surface by Physics-Informed Neural Network. *COMPUTATIONAL ECONOMICS*, 2024.
- Kim, S., Yun, S.B., Bae, H.O., Lee, M. and Hong, Y., Physics-informed convolutional transformer for predicting volatility surface. *QUANTITATIVE FINANCE*, 2024, **24**, 203–220.